Prosit 2 – Machine Learning

De la donnée brute au Perceptron Multi-Couches

Prosit 2 – Machine Learning

Mars 2026

Table des matières

I Préparation des données **2**

1 Objectif **2**

2 Étape 1 – Imputation KNN **2**

3 Étape 2 – Encodage ordinal de `ocean_proximity` **2**

4 Étape 3 – Nettoyage des outliers **2**
4.1 Suppression des prix plafonnés 2
4.2 Suppression des outliers IQR 2

5 Étape 4 – Normalisation Z-Score **3**

6 Schéma du dataset final **3**

II Modélisation – Perceptron Multi-Couches **3**

7 Analyse de colinéarité **3**

8 Séparation Train / Test **4**

9 Architecture du MLP **4**

III Évaluation du modèle **4**

10 Métriques utilisées **4**
10.1 MAE – Mean Absolute Error 5
10.2 RMSE – Root Mean Squared Error 5
10.3 MAPE – Mean Absolute Percentage Error 5
10.4 R^2 – Coefficient de détermination 5

11 Résultats **5**
11.1 Avant nettoyage (20 640 lignes) 5
11.2 Après nettoyage (17 601 lignes) 5
11.3 Comparaison 5

12 Comment lire ces résultats ? **5**

13 Limites identifiées **6**

Première partie

Préparation des données

1 Objectif

Transformer le dataset brut `housing.csv` (20 640 lignes, 10 colonnes) en un fichier prêt pour l'entraînement d'un perceptron : `amputed_normalized_housing_data.csv`.

Pipeline : imputation KNN → encodage ordinal → nettoyage outliers → normalisation Z-Score.

2 Étape 1 – Imputation KNN

La colonne `total_bedrooms` contient 207 valeurs manquantes. On les comble par KNN ($k = 5$, pondération par distance inverse) :

$$\hat{x}_i = \frac{\sum_{j \in \mathcal{N}_k(i)} w_j \cdot x_j}{\sum_{j \in \mathcal{N}_k(i)} w_j}, \quad w_j = \frac{1}{d(i,j)} \tag{1}$$

Les données sont standardisées temporairement avant KNN pour que la distance euclidienne soit cohérente entre colonnes d'échelles différentes, puis dé-standardisées après imputation.

3 Étape 2 – Encodage ordinal de `ocean_proximity`

Catégorie	Valeur
`<1H OCEAN`	0
`INLAND`	1
`ISLAND`	2
`NEAR BAY`	3
`NEAR OCEAN`	4

Table 1 – Encodage ordinal de `ocean_proximity`

Cet encodage est réalisé **avant** l'imputation KNN (KNNImputer requiert des données numériques).

4 Étape 3 – Nettoyage des outliers

Deux types de nettoyage sont appliqués après imputation :

4.1 Suppression des prix plafonnés

965 lignes avec `median_house_value` = 500 001 \$ sont supprimées. Ces valeurs sont **censurées** (le prix réel est supérieur mais non enregistré), ce qui biaiserait l'apprentissage.

4.2 Suppression des outliers IQR

Pour chaque feature numérique (sauf `ocean_proximity` et `median_house_value`), on supprime les lignes dont la valeur est en dehors de l'intervalle :

$$[Q_1 - 1.5 \times IQR,\ Q_3 + 1.5 \times IQR] \tag{2}$$

où $IQR = Q_3 - Q_1$ est l'écart interquartile.

Feature	Outliers	Bornes
`total_rooms`	1 251	$[-1085, 5643]$
`total_bedrooms`	1 227	$[-230, 1175]$
`population`	1 126	$[-629, 3171]$
`households`	1 165	$[-204, 1092]$
`median_income`	364	$[-0.6, 7.7]$

Table 2 – Outliers détectés par feature (méthode IQR)

Bilan : 3 039 lignes supprimées (14.7%), **17 601 lignes restantes**.

5 Étape 4 – Normalisation Z-Score

Toutes les features sauf `median_house_value` (variable cible) sont normalisées :

$$z = \frac{x - \mu}{\sigma} \tag{3}$$

Justification : le perceptron calcule $\hat{y} = f(\mathbf{w}^T\mathbf{x} + b)$. Sans normalisation, les features à grande échelle (`total_rooms` ~ 2000) domineraient le gradient face aux petites (`median_income` ~ 4).

`median_house_value` n'est pas transformée car c'est la cible de la régression.

6 Schéma du dataset final

Colonne	Transformation
`longitude`	Z-Score
`latitude`	Z-Score
`housing_median_age`	Z-Score
`total_rooms`	Z-Score
`total_bedrooms`	KNN + Z-Score
`population`	Z-Score
`households`	Z-Score
`median_income`	Z-Score
`median_house_value`	**Non (cible)**
`ocean_proximity`	Ordinal + Z-Score

Table 3 – Schéma de `amputed_normalized_housing_data.csv` – 17 601 lignes, 10 colonnes

Deuxième partie
Modélisation – Perceptron Multi-Couches

7 Analyse de colinéarité

Avant l'entraînement, on supprime les features redondantes (corrélation $|r| > 0.8$). Pour chaque paire colinéaire, on conserve la feature la mieux corrélée avec la cible.

Feature A	Feature B	$\|r\|$	Supprimée
longitude	latitude	0.925	longitude
total_rooms	total_bedrooms	0.930	total_bedrooms
total_rooms	population	0.857	population
total_rooms	households	0.918	households
total_bedrooms	population	0.878	population
total_bedrooms	households	0.980	total_bedrooms
population	households	0.907	population

TABLE 4 – Paires colinéaires et features supprimées

Features conservées (5) : `latitude`, `housing_median_age`, `total_rooms`, `median_income`, `ocean_proximity`.

Features supprimées (4) : `longitude`, `total_bedrooms`, `population`, `households`.

8 Séparation Train / Test

Le dataset (17 601 lignes après nettoyage) est divisé aléatoirement (`random_state=42`) :

— **Train** : 12 320 échantillons (70%)

— **Test** : 5 281 échantillons (30%)

9 Architecture du MLP

Perceptron multi-couches à 2 couches cachées :

$$\mathbf{5}\ (\text{entrée}) \to \mathbf{64}\ (\text{ReLU}) \to \mathbf{32}\ (\text{ReLU}) \to \mathbf{1}\ (\text{sortie, identité})$$

Propagation avant :

$$h_1 = \text{ReLU}(W_1 x + b_1) \tag{4}$$
$$h_2 = \text{ReLU}(W_2 h_1 + b_2) \tag{5}$$
$$\hat{y} = W_3 h_2 + b_3 \tag{6}$$

Où $\text{ReLU}(z) = \max(0, z)$.

Hyperparamètres :

— Optimiseur : Adam (learning rate = 0.001)

— Loss : MSE (Mean Squared Error)

— Early stopping sur 10% de validation

— Max 500 epochs — convergé en **271 epochs**

— Nombre total de poids : **2 497**

Troisième partie
Évaluation du modèle

10 Métriques utilisées

C'est un problème de **régression**, pas de classification : il n'y a pas de « précision » (*accuracy*) au sens classique. On utilise 4 métriques complémentaires.

10.1 MAE – Mean Absolute Error

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{7}$$

Échelle : dollars ($). Erreur moyenne en valeur absolue — robuste aux outliers.

10.2 RMSE – Root Mean Squared Error

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{8}$$

Échelle : dollars ($). Pénalise les grosses erreurs plus fortement que la MAE (effet quadratique).

10.3 MAPE – Mean Absolute Percentage Error

$$\text{MAPE} = \frac{100}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right| \tag{9}$$

Échelle : pourcentage (%). Erreur relative — indépendante de l'échelle des prix. On peut l'interpréter comme : « en moyenne, le modèle se trompe de X% du prix réel ».

10.4 R^2 – Coefficient de détermination

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2} \tag{10}$$

Échelle : sans unité, entre 0 et 1. Part de la variance du prix expliquée par le modèle. C'est la métrique la plus proche d'une « précision » pour la régression.

11 Résultats

11.1 Avant nettoyage (20 640 lignes)

	MAE ($)	**RMSE ($)**	**MAPE (%)**	**R^2**
Train (70%)	52 455	73 146	29.85	0.60
Test (30%)	52 183	72 690	29.97	0.60

Table 5 – Performance avant nettoyage des outliers

11.2 Après nettoyage (17 601 lignes)

11.3 Comparaison

12 Comment lire ces résultats ?

— **MAE et RMSE en baisse** : le modèle fait des erreurs plus petites en valeur absolue (−12% de RMSE). C'est l'amélioration la plus concrète.

	MAE ($)	**RMSE ($)**	**MAPE (%)**	**R^2**
Train (70%)	45 840	62 841	28.24	0.56
Test (30%)	46 288	63 958	27.98	0.55

TABLE 6 – Performance après nettoyage des outliers

Métrique (test)	**Avant**	**Après**	**Amélioration**
MAE ($)	52 183	46 288	−11.3%
RMSE ($)	72 690	63 958	−12.0%
MAPE (%)	29.97	27.98	−2.0 pts
R^2	0.60	0.55	−0.05

TABLE 7 – Impact du nettoyage des outliers sur la performance

— **MAPE en baisse (28%)** : en moyenne, la prédiction s'écarte de 28% du prix réel, soit une « précision relative » d'environ **72%**.
— **R^2 légèrement plus bas (0.55 vs 0.60)** : normal — en supprimant les outliers, on réduit la variance totale des prix. Le dénominateur de R^2 diminue, donc le ratio peut baisser même si les erreurs absolues sont plus petites. La MAE et le RMSE sont de meilleurs indicateurs ici.
— **Train ≈ Test** : pas de sur-apprentissage.

13 Limites identifiées

— 4 features supprimées par colinéarité — un modèle non-linéaire pourrait en tirer plus d'information.
— Seules 5 features utilisées — des données supplémentaires (surface, nombre de pièces, quartier) amélioreraient la prédiction.
— Le seuil IQR (1.5×) est un choix conservateur ; un seuil plus souple (2× ou 3×) garderait plus de données.